

Contents lists available at [ScienceDirect](#)

Process Safety and Environmental Protection

journal homepage: www.elsevier.com/locate/psep


Numerical modelling of gas dispersion using OpenFOAM

Juliane Fiates, Sávio S.V. Vianna*

University of Campinas – UNICAMP, Cidade Universitária “Zeferino Vaz” Av. Albert Einstein, 500, CEP 13083-852 Campinas, SP, Brazil

ARTICLE INFO

Article history:

Received 22 February 2016

Received in revised form 31 August 2016

Accepted 9 September 2016

Available online 17 September 2016

Keywords:

Computational fluid dynamics (CFD)

Jets

Gas dispersion

OpenFOAM

ANSYS-CFX

Open source code

Consequence modelling

ABSTRACT

In the current work the rhoReactingBuoyantFoam solver was customised for performing gas leak and gas dispersion modelling. Using experimental data from gas leaks the proposed modelling was investigated for subsonic and sonic releases. The gas molar fraction and velocity decay along the jet centreline were calculated using the modified reacting solver, and the numerical findings were compared with available experimental data. Different approaches for the turbulence closure problem were considered using standard two-equation models. The numerical stability of the solver was also investigated varying the CFL number for a set of simulations. The work also considered the modelling of gas cloud volume in a real engineering case. Standard computational setup for ANSYS-CFX was applied, and the same set of scenarios were modelled in OpenFOAM using the modified rhoReactingBuoyantFoam solver. The analysis considered 5 different leak directions and 4 wind directions in a typical industrial site.

For all scenarios simulated, very good agreement with experimental data and with the commercial CFD (computational fluid dynamics) tool considered in this study was observed. The results are within 10% tolerance intervals. Detailed information of the modelling is also provided, which enable any CFD user to reproduce the results and also apply it for future analysis.

© 2016 Institution of Chemical Engineers. Published by Elsevier B.V. All rights reserved.

1. Introduction

Accidental releases of flammable or toxic gas can significantly drive the risk level to workers and the environment.

Risk management relies on the proper understanding of the hazards and the risk gas releases pose to personnel. It is therefore crucial to perform a large amount of failure scenarios in order to estimate the measures of protection such as gas detectors location and reinforcement structures due to accidental explosions caused by gas releases.

There have been a large number of experiments concerning gas dispersion. Some of these experiments comprising jets (Birch et al., 1984, 1987; Chuech et al., 1989; Wakes et al., 2002; Rocco and Woods, 2015) and others dealing with large-scale

field trials such as MUST, Kit Fox and Praine Grass (Hanna et al., 2004; Hanna and Chang, 2001). They provide data that enable the development of new mathematical dispersion models, and they also help the validation process of numerical tools.

On the other hand, the numerical modelling appears as an alternative approach to create dispersion scenarios. Computational fluid dynamics (CFD) simulation deals with complex geometries, and it is able to produce a wide range of accidental scenarios at low cost (Qiao and Zhang, 2010; Tauseef et al., 2011).

As far as CFD dispersion modelling is concerned, various commercial tools are extensively used as they present a friendly graphic interface and they are widely accepted in both academic and industrial sectors. Concerning the gas jet

* Corresponding author. Fax: +55 19 3521 3910.

E-mail address: savio@feq.unicamp.br (S.S.V. Vianna).

<http://dx.doi.org/10.1016/j.psep.2016.09.011>

0957-5820/© 2016 Institution of Chemical Engineers. Published by Elsevier B.V. All rights reserved.

simulation, many turbulence models were tested and validated using commercial tools (e.g. CFX, Fluent) (Rigas and Sklavounos, 2004; Gant and Iving, 2005; Wilkening et al., 2008). Far field gas plumes simulation were reported in Qiao and Zhang (2010) work where the authors create a method to quantify flammable gas cloud using FLACS. Hansen et al. (2010) also validated FLACS code against experimental data based on the procedure of the Model Evaluation Protocol (MEP). Cormier et al. (2009) analyzed the ANSYS-CFX code to perform consequence analysis for LNG releases. Initially, a validation of the tool against experimental data was considered and the numerical findings presented good agreement with experimental data. Additionally, key parameters of discharge source and dispersion variables such as LNG pool area, evaporation rate, wind velocity and obstacles effect also were simulated.

Currently, OpenFOAM has also been used to simulate gas dispersion. Mack and Spruijt (2013) applied OpenFOAM code to heavy gas dispersion calculation. For large scale simulation, OpenFOAM results were compared with ANSYS-Fluent code. For some cases, the modelling performed with OpenFOAM presented better results than ANSYS-Fluent when compared with experimental data. The comparison, however, was limited to gravity flows and it did not consider the jet release near-field region where the momentum is high.

Similarly, Dixon (2012) has applied PDRFoam solver for gas dispersion simulation. Several simulations were conducted and the flammable gas cloud volume was compared with experimental data. Most of OpenFOAM results fall within a 10% confidence interval.

Kumaresh et al. (2016) customised an OpenFOAM solver to deal with gas dispersion calculation. The solver, namely cloudIgnitionFoam, comprises a transient leak discharge model, gas dispersion and ignition probability models. Although some results are presented, the code has not been validated against existing experimental data.

There have also been attempts to use FDS (Fire Dynamics Simulator) for gas dispersion calculations. Mouilleau and Champassith (2009) report the evaluation of the code based on comparison with widely known experimental sets as Falcon and MUST. The code also was tested against PHAST (Process Hazard Analysis Software Tool) software. The modelling, however, considered only low momentum gas leaks, which are rare in accidental releases where the reservoir storage is kept at high pressure. On the other hand, since FDS relies on the LES (Large Eddy Simulation) approach to capture the fluctuations due to turbulence, Mouilleau and Champassith (2009) claim better results when the wind velocity fluctuations are considered in the modelling.

More recently Ferreira and Vianna (2014, 2016) developed a source model, namely DESQr (Diameter of Equivalent Simulation for Quicker Run) that was implemented in the framework of FDS (Fire Dynamics Simulator) to model high momentum jet release and gas dispersion. The results obtained with the modified FDS code agree well with experimental data and ANSYS-CFX for jet release and gas dispersion data. However, since the code relies on LES approach the computational time is an issue when compared with the computational time spent using RANS (Reynolds Averaged Navier Stokes) modelling.

In this work we explore the complementary application of OpenFOAM tool applied to gas dispersion using another solver, namely rhoReactingBuoyantFoam.

Firstly, we present the solver rhoReactingBuoyantFoam and the customisation executed. The modelling of the gas jet release is carefully investigated, and two different

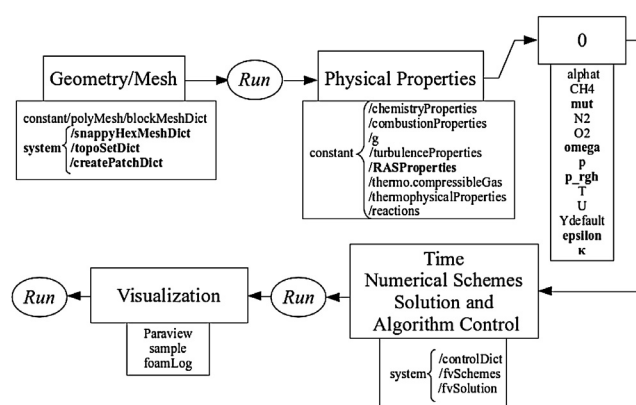


Fig. 1 – Flowchart with all steps of the gas dispersion modelling considered in the current research.

turbulence models are considered. The results are compared with experimental data alongside the mesh sensitivity analysis and optimised time step. A jet simulation performed by Gant and Iving (2005) is reproduced and good agreement is observed. An engineering case study was developed based on a typical industrial facility. The analysis considered 5 leak directions and 4 wind directions. Numerical findings are compared with a commercial tool ANSYS-CFX and no significant difference is observed. We concluded with a discussion of our work with the implication for applying an alternative open source CFD tool on gas dispersion modelling considering the high momentum release. The Appendix A also provides detailed information of the modelling. It allows for the verification of the results obtained in this work, and it also serves as a guideline for future simulations.

2. Gas dispersion modelling

As far as gas dispersion is concerned the large scale objects play a more significant role than the small scale parts of the geometry, such as piping and small bore fittings. The former, on the other hand, is very important in an accidental explosion. The current analysis is mainly focused on the influence of the large scale geometry. Therefore smaller scale objects have not been considered. Having said that, OpenFOAM has a native mesh generator (blockMesh) and it is able to deal with complex CAD geometries through the snappyHexMesh utility.

The gas dispersion was modelled using the solver rhoReactingBuoyantFoam. The conservation equations modelling the transport of chemical species were applied. The convective term was discretised using the upwind scheme. The diffusion term used central difference scheme. The time advancement was addressed using first order Euler. Pressure – velocity coupling is addressed via PIMPLE algorithm.

Although the solver was initially developed for reacting flows, the chemical reaction source term in the chemical species conservation equation was disabled. In order to accomplish the modelling of the gas dispersion, all physical properties concerning reactions were switched off. The chemical components were set as inert in the ChemistryProperties file located in the constant folder as listed in Fig. 1.

The density, rho, is selected as function of the solver and the reaction file (reactions) does not consider any reaction. In the current research, two turbulence closure models were investigated. The default set up for the $\kappa - \epsilon$ and SST $\kappa - \omega$ are found in the turbulenceProperties file.

Fig. 1 presents a detailed flow of information as considered in the current work.

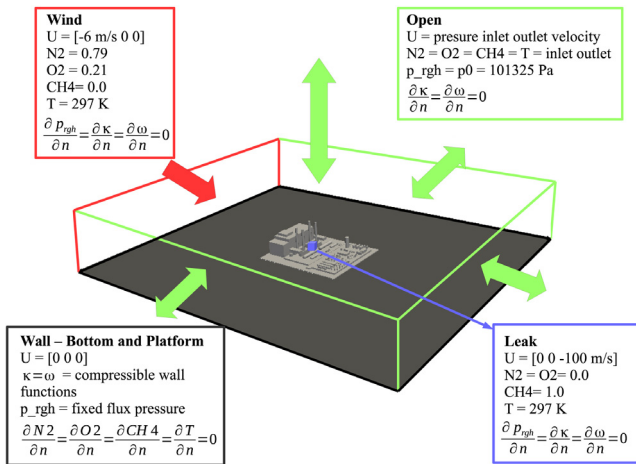


Fig. 2 – Computational domain and the groups of boundary conditions applied in the simulation using the rhoReactingBuoyantFoam package.

The boundary conditions were set up in the 0 (zero) file. All variables of interest for the boundary conditions are shown in Fig. 1.

Fig. 2 shows the geometrical model as well as the computation domain and boundaries considered in the analysis. Analysis of Fig. 2 shows that the boundary conditions are grouped into four categories: wall, open, wind and leak. For each category the following set up was considered:

- Wall: Zero flux through the wall. Non-slip conditions.
- Open: Zero gradient and prescribed pressure.
- Wind: Prescribed inlet velocity.
- Leak: Prescribed gas velocity for given hole size.

3. Results

3.1. The near field jet simulation

Birch et al. (1984, 1987) conducted a series of experiments to evaluate the concentration field of natural gas over the pressure range from 2 to 70 bar. The apparatus used in the experiment considered a 2.7 mm diameter hole and methane being released between 92 and 92.4% as natural gas and a mean molecular weight of 17.32.

Two experiments considering the upstream pressure of 2.0 bar and 3.5 bar were modelled using the numerical approach discussed in this work. For both cases, the computational simulation was performed in a 1 m (H) $\times$ 0.1 m (W) $\times$ 0.05 m (L) domain. The computational domain and the mesh is shown in Fig. 3. The cases modelled in OpenFoam were set as; Reynolds number of 7.0×10^4 and Mach number of 0.97 and Reynolds number of 8.7×10^4 and Mach number of 1.27 for the back-pressure of 2.0 bar and 3.5 bar, respectively.

The computing of the cases described in the experiment set up was conducted using RANS approach. The influence of the turbulence model on the results was investigated considering the SST κ - ω model and the κ - ϵ model.

Three mesh sizes were considered. The most refined grid comprised 37,628 computational cells, namely Grid.03. The intermediate and the coarsest meshes comprised 20,496 and 11,422 computational cells, namely Grid.02 and namely Grid.01, respectively.

Fig. 4(a) and (b) shows the methane molar fraction decay as a function of the non-dimensional distance from the leak

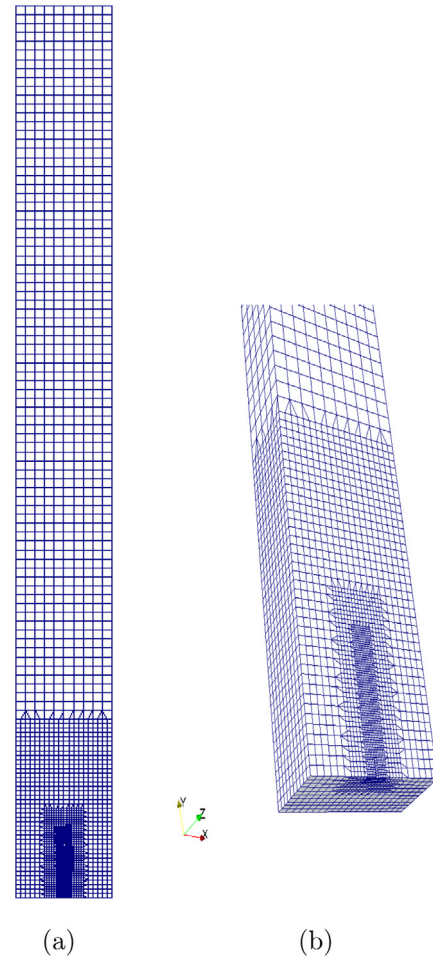


Fig. 3 – Computational mesh used in the current work. (a) Details of the refined mesh region close to the gas leak. (b) 3D cut in the centre of the leak orifice.

orifice for the two cases considered (upstream pressure of 2.0 bar and 3.5 bar.). The y-axis shows the methane molar fraction and the x-axis shows the distance from the leak orifice normalised by the leak diameter. The results are presented for three different mesh sizes. Analysis of the plots shows that the results are very similar for all meshes considered in the investigation.

Numerical results for the optimal mesh size were also compared with the experimental data reported by Birch et al. (1984). The comparison is presented in Fig. 4(c) and (d). Analysis of the plot shows good agreement between the numerical results and experimental data for the case considering the upstream pressure of 2.0 bar (Fig. 4c) and also for the case considering upstream pressure of 3.5 bar (Fig. 4d).

The same experimental apparatus previously introduced was also applied for the investigation of air jets. The results were reported by Birch et al. (1987). The experimental data was used in order to investigate the behaviour of the turbulence model in the jet release.

Considering the same set of upstream pressure used in the methane analysis, the velocity decay of the jet as function of the non-dimensional distance was investigated.

The simulations considered the SST κ - ω and κ - ϵ turbulence model. Numerical findings have also been compared with experimental data by Chuech et al. (1989). Fig. 5 shows the non-dimensional velocity on the y-axis and the non-dimensional distance on the x-axis. Fig. 5(a) compares the numerical results with the experimental data considering the SST κ - ω and κ - ϵ

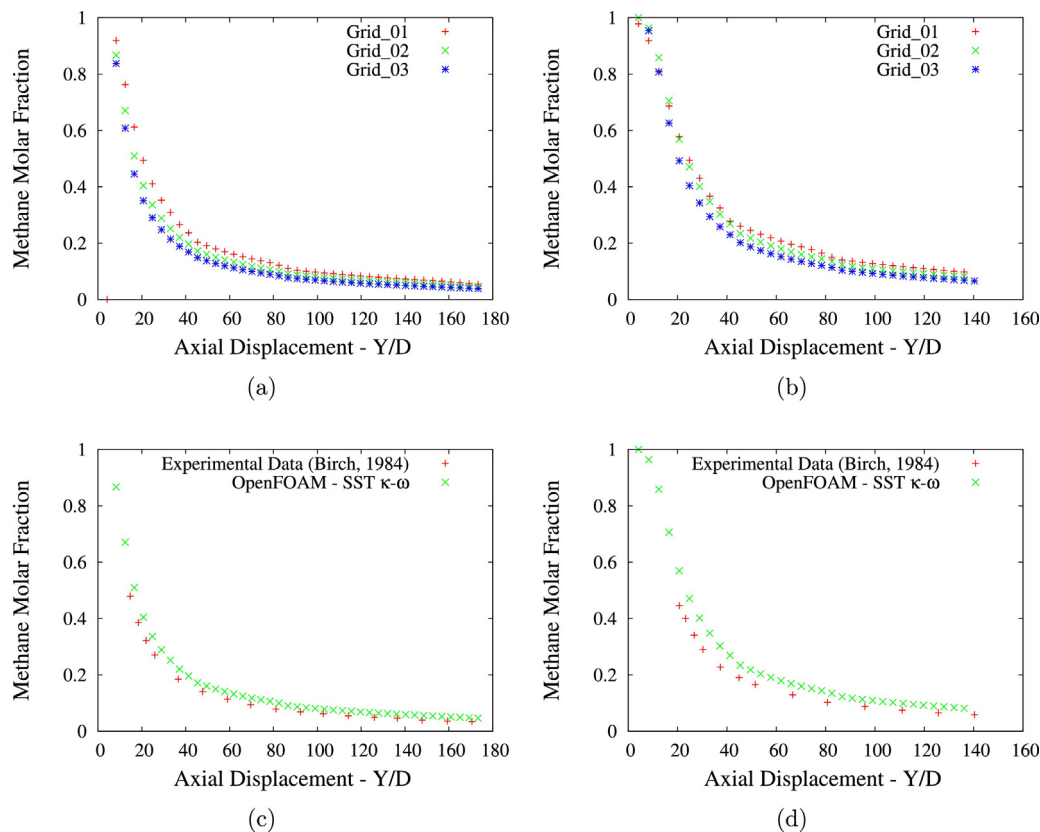


Fig. 4 – Methane molar fraction decay (Y = axial distance, D = orifice diameter): (a) Upstream pressure of 2.0 bar and (b) upstream pressure of 3.0 bar. (c) Comparison with experimental data for upstream pressure of 2.0 bar. (d) Comparison with experimental data for upstream pressure of 3.0 bar.

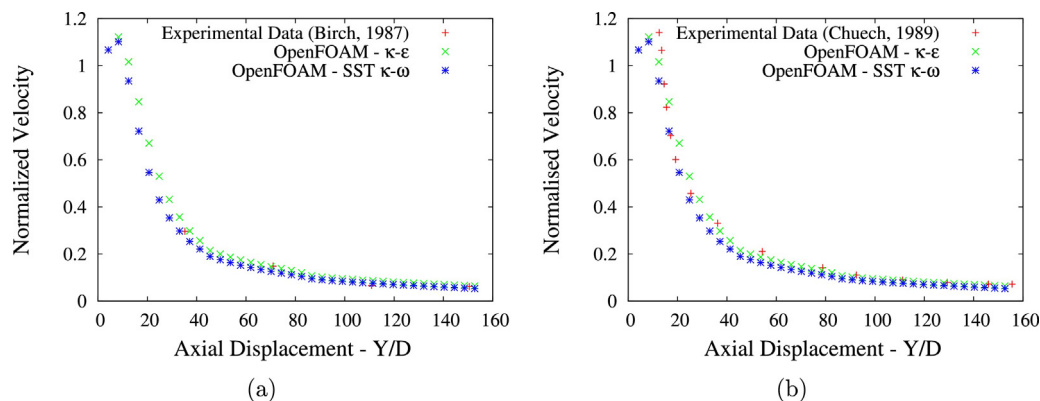


Fig. 5 – Sonic air jet velocity decay by dimensionless ratio (Y = axial distance, D = orifice diameter): (a) Comparison with Birch et al. (1987) and (b) comparison with Chuech et al. (1989).

turbulence model. Fig. 5(b) presents the same analysis of Fig. 5(a). The experimental data presented in Fig. 5(b) is due to Chuech et al. (1989). The experiment conducted by Chuech et al. (1989) is similar to the experiment performed by Birch et al. (1987). The main difference is the diameter of the orifice. The later used 9.5 mm of diameter for the leak while the former considered 2.7 mm.

Analysis of the plots presented in Fig. 5 shows that there seems to be no significant influence of the turbulence model on the results obtained.

3.2. Courant number

The Courant number influence was investigated for the coarsest grid in which the numerical solution was independent of the mesh size.

The normalised velocity as a function of the non-dimensional distance from the leak orifice is presented in Fig. 6. The numerical results are compared with experimental data for the three Courant number values investigated.

Fig. 6(a) presents results for the case in which air jet experiments were conducted by Birch et al. (1987). Fig. 6(b) shows the experimental results documented by Chuech et al. (1989). Analysis of both plots shows little dependency on the Courant number.

Fig. 6(c) presents the residual error for each Courant number tested. Analysis of all plots in Fig. 6 shows little dependency on the Courant number. On the other hand, higher Courant numbers can lead to numerical stability issues and the simulation may not converge. The optimal Courant number for the cases investigated was 10.0.

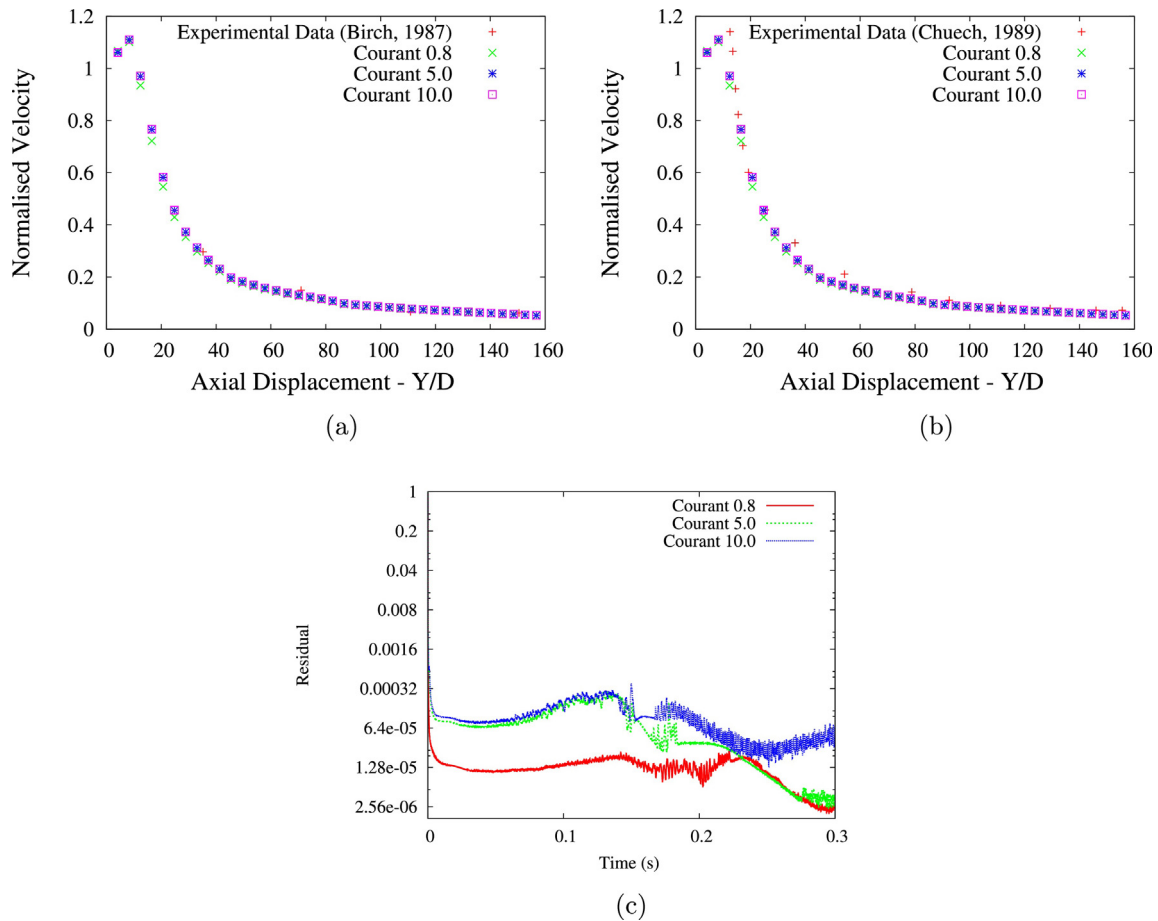


Fig. 6 – Analysis of numerical time step influence in the velocity decay of sonic air jet: (a) Comparison with Birch et al. (1987), (b) comparison with Chuech et al. (1989) and (c) residue from the axial velocity by simulated time step.

Table 1 – Influence of the Courant number in the computational time for the optimal grid size. Two releases conditions (upstream pressure of 2.0 bar and 3.5 bar) are considered. Results are presented for air and methane releases.

	Mesh	Courant	Simulation time (h)
Air	Upstream pressure 2.0 bar	Grid.02	0.8
		5.0	5.04
		10.0	0.57
	Upstream pressure 3.5 bar	Grid.02	0.8
		5.0	11.82
		10.0	0.71
Methane	Upstream pressure 2.0 bar	Grid.02	0.8
		5.0	5.30
		10.0	1.33
	Upstream pressure 3.5 bar	Grid.02	0.8
		5.0	8.02
		6.0	2.12

On the other hand, analysis of Table 1 shows significant reduction in the computational time for the three values of the Courant number considered in the study. For the cases considering air as the material released and upstream pressure of 2.0 bar, the simulation time has been reduced 5 times for the intermediate value of the Courant number. For the case where the upstream pressure was 3.5 bar, the reduction in the simulation time when the intermediate value of the Courant number was considered was around 11 times.

The same trend in the reduction of the simulation time was observed for the cases in which methane was considered as the material released.

3.3. Flammable gas cloud

Gant and Ivings (2005) conducted a jet simulation using ANSYS-CFX. The case modelled using CFD considered a release from an orifice of 10.5 mm of diameter. The stagnation pressure was 5.0 bar and the exit velocity was set to be 375 m/s. The release temperature was 250 K.

The same setup was applied in OpenFOAM considering the modifications in the code as previously discussed.

The cloud volume calculated by Gant and Ivings (2005) was 0.0275 m³. The result obtained when using the numerical modelling discussed in this work was 0.0245 m³. It is important to stress that the computational meshes applied in the simulation were not the same. Although the mesh might have influenced the result, the difference between the volume is considered to be very small.

Fig. 7 below shows a plane cut through the middle of the jet simulated.

3.4. Engineering case

Gas dispersion scenarios were considered in a typical offshore topside. The process deck was 104 m wide by 122 m long. The accommodation module was the largest building on the deck and it was 38.5 m high (Ferreira and Vianna, 2014). Some simplifications in the CAD model can be adopted, since the small

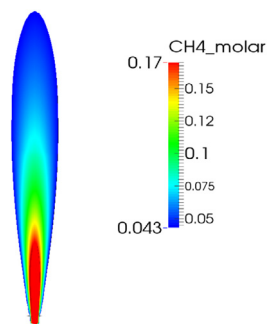


Fig. 7 – Jet simulation in accordance with the computational set up reported by Gant and Ivings (2005). Gas volume calculated with the proposed model is 0.0247 m^3 against 0.0275 m^3 calculated by ANSYS-CFX.

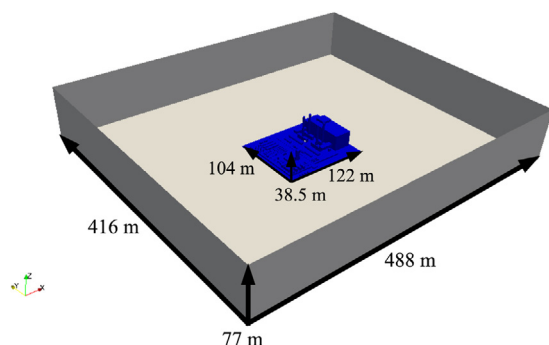


Fig. 8 – Platform dimensions and the computational domain used in the simulation.

scale geometry is not as important as in explosion studies (Kumaresh et al., 2016).

The near field region of the jet was modelled in order to avoid the compressibility effects. Since the jet momentum falls fairly quick as a function of the distance from the release, a new set of boundary conditions was applied. It considered the expanded diameter and 100 m/s jet speed (Mach number lower than 0.3). The exit conditions were calculated assuming an isentropic expansion from a reservoir at 20 bar and 300 K. The leak hole size was set as 10" and the expanded diameter was calculated as suggested by Birch and Schefer and reported by Benintendi (2010).

The computational mesh was generated as hexagonal elements using OpenFOAM native mesher. The computational domain was set to be twice the length of the geometry in x and y directions. The z direction was one time the height of the accommodation module. Fig. 8 shows the computational domain applied in the simulations. The leak was placed on the centre of the process deck as shown in Fig. 9. Four different wind directions were considered for each leak orientation.

The leak placement and orientation ($-z$) was based on the worst case scenario for mesh investigation. It follows previous investigations for the largest cloud in congested areas (Ivings et al., 2010). It also addresses the fact that critical situations should be raised in gas dispersion analysis for explosion calculation purposes according to Qiao and Zhang (2010) and Tauseef et al. (2011).

Mesh sensitivity analysis for all scenarios previously described was performed. The same gas dispersion scenarios were simulated using ANSYS-CFX. The number of elements in each mesh is presented in Table 2.

Fig. 10 shows the computational mesh used in the simulations. The mesh considered in the OpenFOAM cases

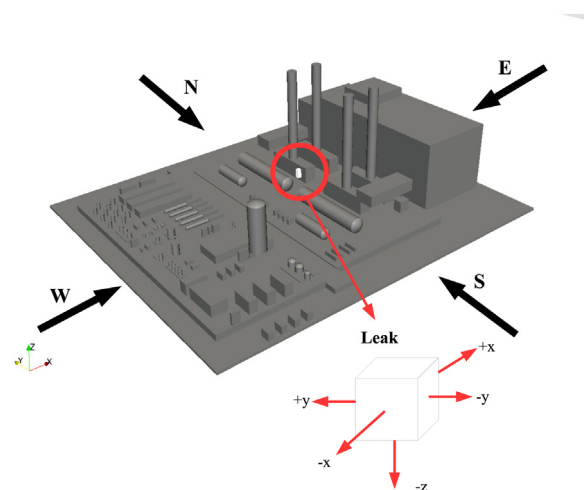


Fig. 9 – Wind direction and location of the leak source on the deck.

Table 2 – Computational mesh size used in OpenFOAM and ANSYS-CFX.

Code	Mesh	Cells number
OpenFOAM	Mesh_01	375,834
	Mesh_02	1,301,663
	Mesh_03	1,843,444
ANSYS-CFX	Mesh_01	1,038,199
	Mesh_02	3,624,035
	Mesh_03	7,656,966

comprised 1,301,663 cells and it is shown in Fig. 10(a). Fig. 10(b) shows the mesh used in ANSYS-CFX. It comprised 3,624,035 cells.

The OpenFOAM mesh is structured and hexahedral as previously mentioned. The ANSYS-CFX mesh is unstructured and tetrahedral. Fig. 10(c) and (d) shows the 3D mesh and the cross section (y) of the meshes.

In order to investigate how the results calculated by OpenFOAM and ANSYS-CFX compare, the mesh generation procedure for each software was strictly followed. The refinement of the mesh was performed based on the comparison with experimental data for the near field simulation. The different features of the meshes as well as the refinement regions considered led to significant difference in the total number of computational cells.

The boundary conditions for each software are described in Table 3. The wind speed applied was 6 m/s. It has been selected based on average wind speed at Campos basin, Rio de Janeiro, Brazil.

Table 4 details the numerical set up for both softwares considered in the analysis. The simulations were run using one CPU by case in a computer with a processor Intel® Core™ i7 vPro™ and 8 GB of RAM memory. The OpenFOAM simulations took around 168 h to compute 800 s of physical time. The time step was governed by the maximum Courant number of 10.0. The ANSYS-CFX simulation took around 4 h to converge.

Fig. 11 shows the flammable gas cloud calculation using ANSYS-CFX for three different mesh sizes. The y-axis shows the volume cloud at steady condition and the x-axis shows the mesh categories as described in Table 2. Analysis of the plot presented in Fig. 11 shows no significant difference in the cloud volume for Mesh_02 and Mesh_03. The simulations were performed with the wind blowing from East in accordance

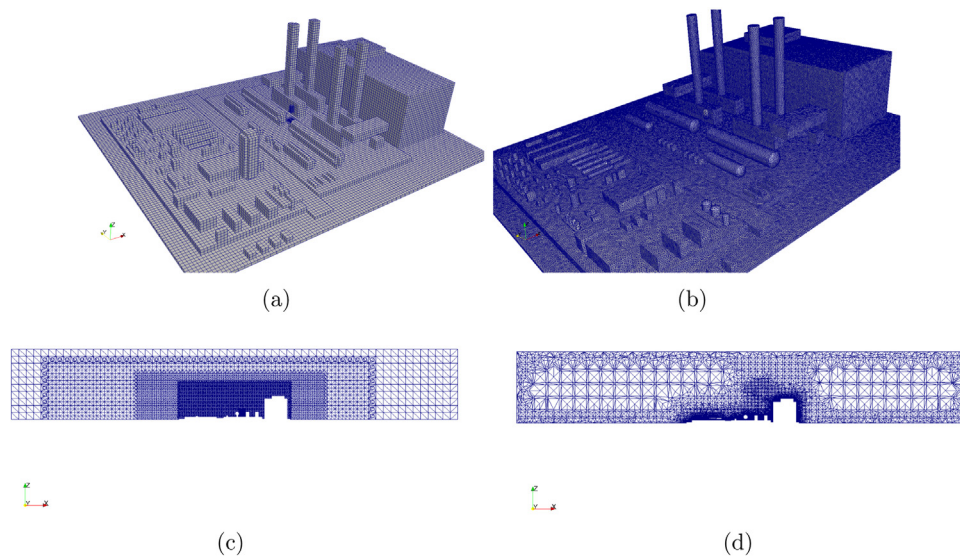


Fig. 10 – Mesh used in the simulation of the offshore platform – (a) Platform 3D structure – OpenFOAM mesh, (b) Platform 3D structure – ANSYS-ICEM mesh, (c) Sectional view – OpenFOAM mesh and (d) Sectional view – ANSYS-ICEM mesh.

Table 3 – Boundary conditions applied in the gas dispersion simulation using OpenFOAM and ANSYS-CFX.

Region	Boundary conditions in OpenFOAM and ANSYS-CFX	
	OpenFOAM	ANSYS-CFX
Solid surfaces	Wall function (no slip condition)	No slip condition
Leak	Prescribed velocity	Prescribed mass discharge rate
Wind	Prescribed velocity (uniform flow)	Prescribed velocity (uniform flow)
Computational domain	Total pressure $P_0 = 101,325$ Pa	Relative pressure = 0

Table 4 – Set up parameters applied in the gas dispersion simulation using OpenFOAM and ANSYS-CFX.

Parameter	Value or type adopted in each software	
	OpenFOAM	ANSYS-CFX
Reference pressure	1 atm	1 atm
Regime flow	Unsteady and compressible	Steady and incompressible (subsonic)
Turbulence modelling	SST $\kappa - \omega$	SST $\kappa - \omega$
Interpolation scheme	Upwind	Upwind
Numerical time step	Adjustable for a maximum Courant (10.0)	Auto timescale (1.0)
Convergence criteria	Residual	RMS
Maximum residual	$1 \cdot 10^{-6}$	$1 \cdot 10^{-6}$
Final time/minimum and maximum number of iterations	Cloud volume to get in regime	1–1000

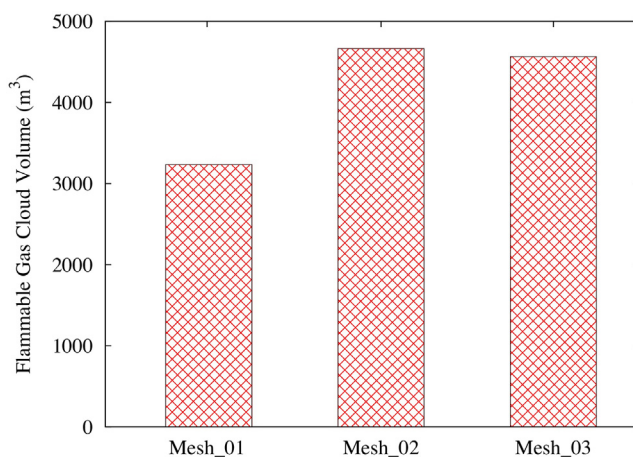


Fig. 11 – Mesh sensitivity analysis. Wind is blowing from East taking into account the wake effect generated by the largest obstruction. The case was simulated using ANSYS-CFX.

with the orientation presented in Fig. 9. The wind direction for the mesh analysis was selected aiming to assess the wake effect generated by the largest obstruction on the topside.

Mesh sensitivity analysis was previously conducted by Ferreira and Vianna (2014) considering the same geometry. Therefore for the current study only one wind direction was considered.

Fig. 12 presents the mesh analysis for the OpenFOAM simulations. Since the current research was mainly focused on the performance of the combustion OpenFOAM solver applied for gas dispersion modelling, four wind directions were considered in the sensitivity analysis. The computational mesh size considered in the mesh sensitivity analysis is presented in Table 2.

The flammable gas volume difference between Mesh.01 and Mesh.03 for all direction was about 30%. Considering the results calculated using Mesh.02 and Mesh.03 the difference was found to be around 11%. The verification of the final volume of the cloud was performed by monitoring the volume

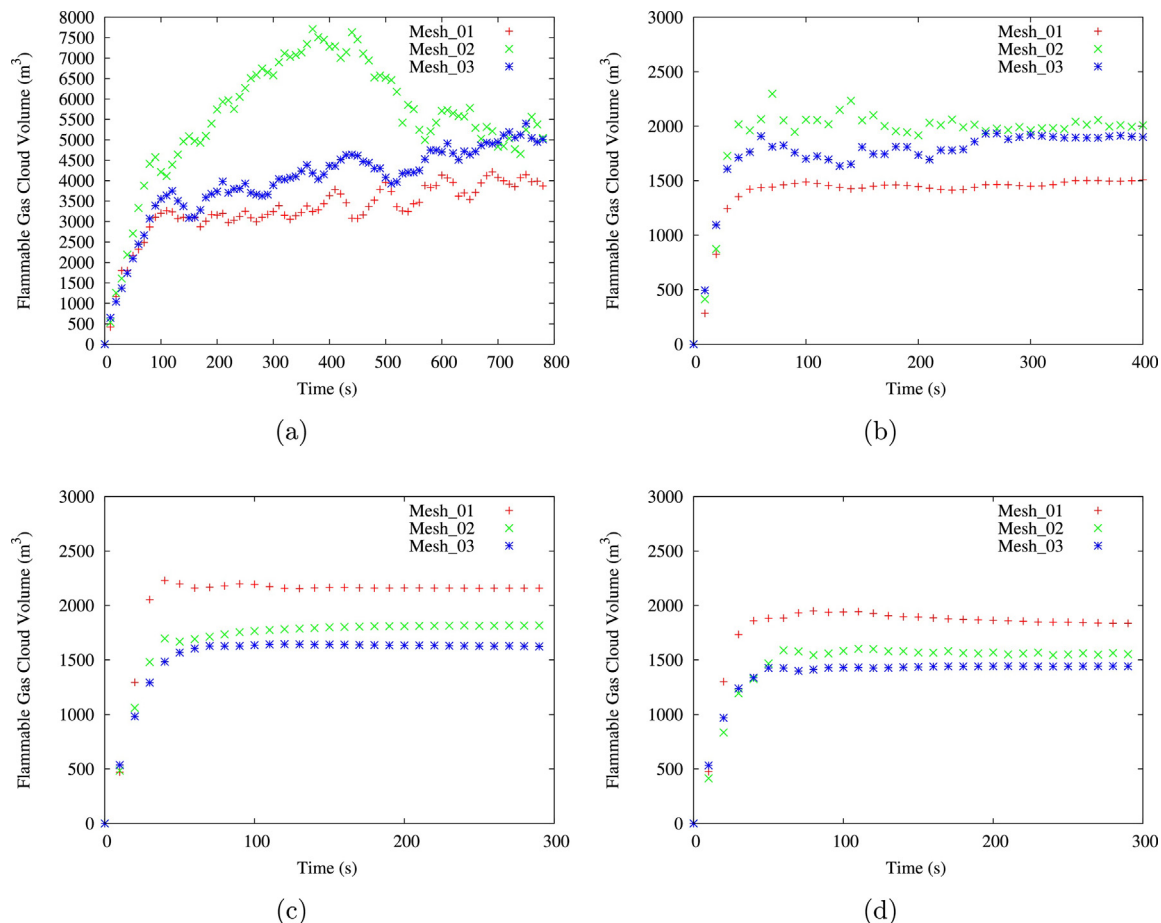


Fig. 12 – Mesh sensitivity analysis for simulation of the offshore platform performed in OpenFOAM – Evolution of the volume of the flammable cloud over time: (a) East, (b) West, (c) North and (d) South.

Table 5 – Flammable gas cloud volume – OpenFOAM versus ANSYS-CFX.

Leak	Wind	Gas cloud volume (m ³)		Difference (%)
		OpenFOAM	ANSYS CFX	
+x (East)	East	3345.4	3558.4	6.0
	West	1127.5	1280.1	11.9
	North	2100.0	1778.0	18.1
	South	2044.0	1833.1	11.5
–x (West)	East	1554.2	1469.7	5.7
	West	1513.0	1592.5	5.0
	North	1000.1	899.5	11.2
	South	1204.0	1042.7	15.5
+y (North)	East	908.3	886.1	2.5
	West	866.7	831.9	4.2
	North	1495.7	1634.5	8.5
	South	990.8	885.8	11.8
–y (South)	East	1003.2	963.0	4.0
	West	941.1	1000.7	5.9
	North	1114.3	935.4	19.1
	South	1350.5	1466.5	7.9
–z (Down)	East	5011.1	4662.5	7.5
	West	1901.0	1959.8	3.0
	North	1624.3	1714.2	5.2
	South	1441.2	1409.0	2.3

until it reaches the steady state, as shown in Fig. 12. The y-axis shows the flammable gas cloud volume and the x-axis shows the elapsed time from leak start. Analysis of the plots presented in Fig. 12(a)–(d) shows that there is no significant

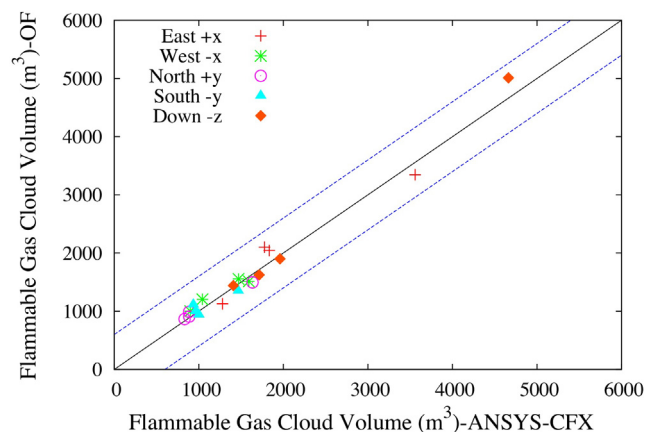


Fig. 13 – Flammable gas cloud volume comparison between OpenFOAM and ANSYS-CFX. Dashed lines represent 10% tolerance interval.

difference between the cloud volume calculated using Mesh.02 and Mesh.03.

Fig. 12(a) shows a different behaviour for the intermediate mesh. It is important to bear in mind that the wind blowing from East interacts with the mains obstruction in the process area. The separation of the boundary layer leads to a recirculation pattern at the wake of the building. Increased mixing leads to dilution of the cloud and the flammable cloud volume wiggles a little bit. However, at 800 s the difference is not significant, particularly for gas dispersion modelling.

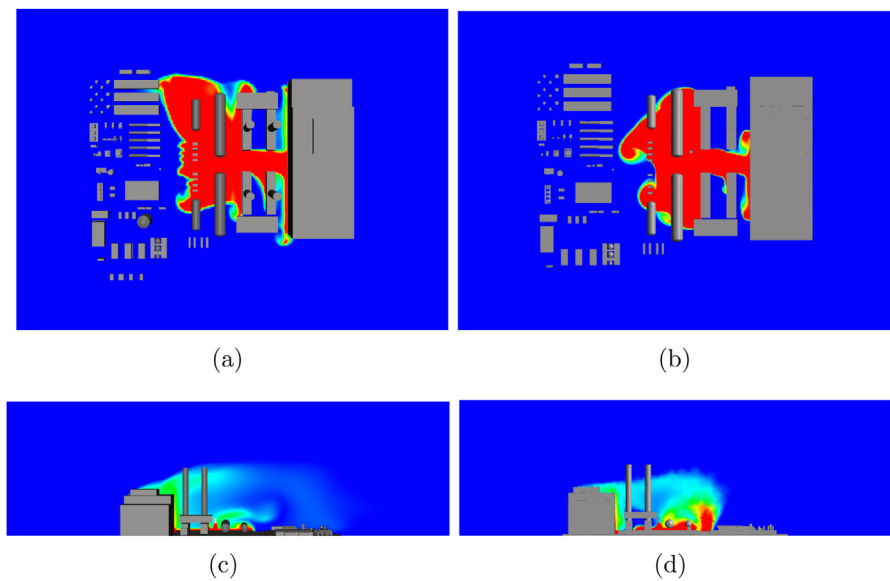


Fig. 14 – Flammable gas cloud contours. Wind is blowing from East and leak points downwards. (a) Top sectional view – OpenFOAM, (b) Top sectional view – ANSYS-CFX, (c) Side sectional view – OpenFOAM, (d) Side sectional view – ANSYS-CFX. All plots at 800 s.

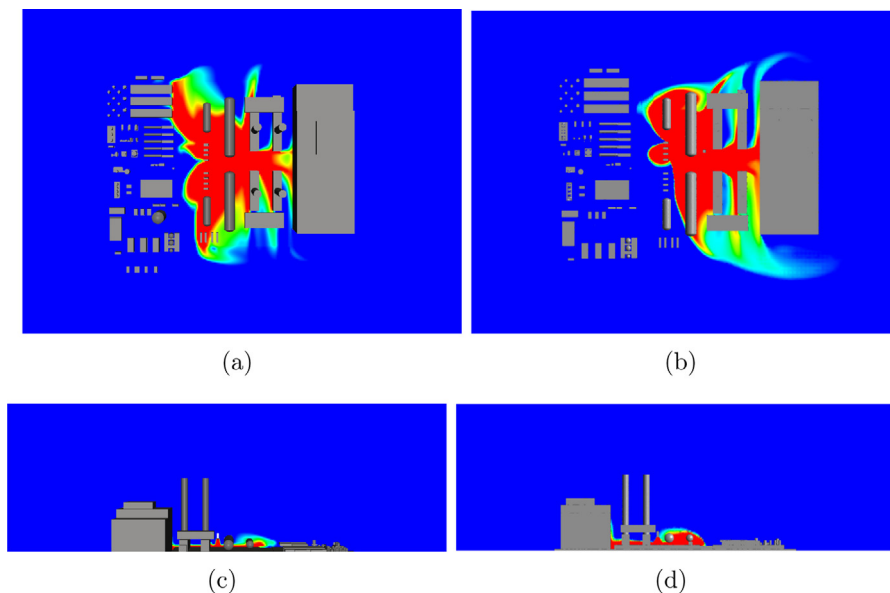


Fig. 15 – Flammable gas cloud contours. Wind is blowing from West and leak points downwards. (a) Top sectional view – OpenFOAM, (b) Top sectional view – ANSYS-CFX, (c) Side sectional view - OpenFOAM, (d) Side sectional view – ANSYS-CFX. All plots at 400 s.

Fig. 13 compares the flammable gas cloud calculated using OpenFOAM with the results calculated using ANSYS-CFX for all combination of leak and wind directions shown in Fig. 9. The solid line in the plot represents identical results. The dashed lines above and below the solid line represents the 10% tolerance intervals. The cloud volume calculated with OpenFOAM are shown on the y-axis and the results calculated using ANSYS-CFX are shown on the x-axis.

Fig. 13 compiles all results. Analysis of Fig. 13 shows that the results obtained using OpenFOAM are very close to those calculated by ANSYS-CFX.

Table 5 shows the gas cloud volumes calculated by OpenFOAM and ANSYS-CFX. The difference between the results is also presented. As expected the scenario with wind blowing from East wind and Down leak (–z) generated the largest cloud

in the process area due to the recirculation caused by the wake effect of the largest building. This particular case presented a difference of 7.47% between the results. For all other cases considered in the analysis the difference of the results is within 20%.

Fig. 14 shows the gas dispersion contour for the wind blowing from East wind and Down leak direction. Top plots shows the x–y view and the bottom plots shows z–x view. Fig. 14(a) shows the results obtained using OpenFOAM and Fig. 14(b) shows the results obtained using ANSYS-CFX. Analysis of Fig. 14(a) and (b) shows that the cloud formed engulfs half of the process topside. There seems to be small differences in gas flow pattern in the process deck. Overall, from the qualitative point of view the results are similar. The minor differences noticed in the plots are due to the mesh. It is important to

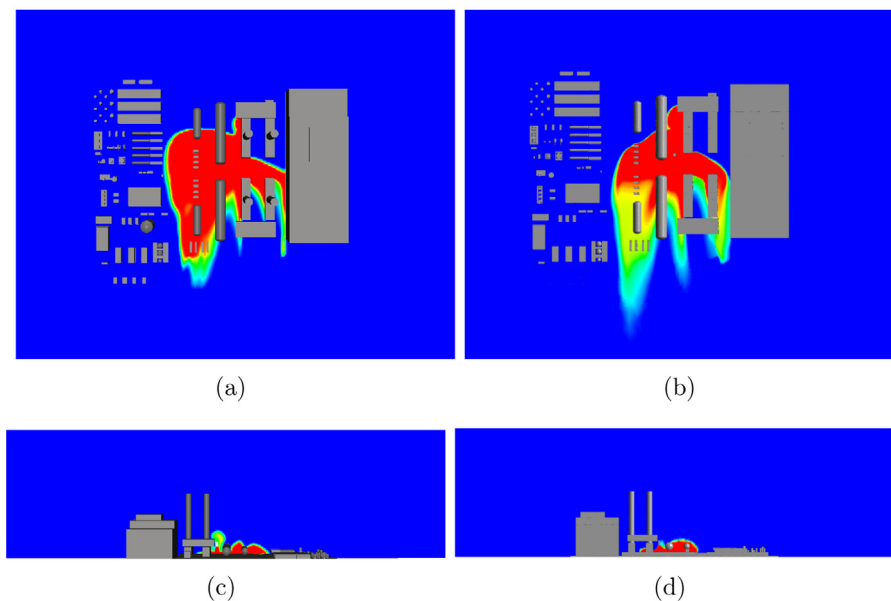


Fig. 16 – Flammable gas cloud contours. Wind is blowing from North and leak points downwards. (a) Top sectional view – OpenFOAM, (b) Top sectional view – ANSYS-CFX, (c) Side sectional view – OpenFOAM, (d) Side sectional view – ANSYS-CFX. All plots at 300 s.

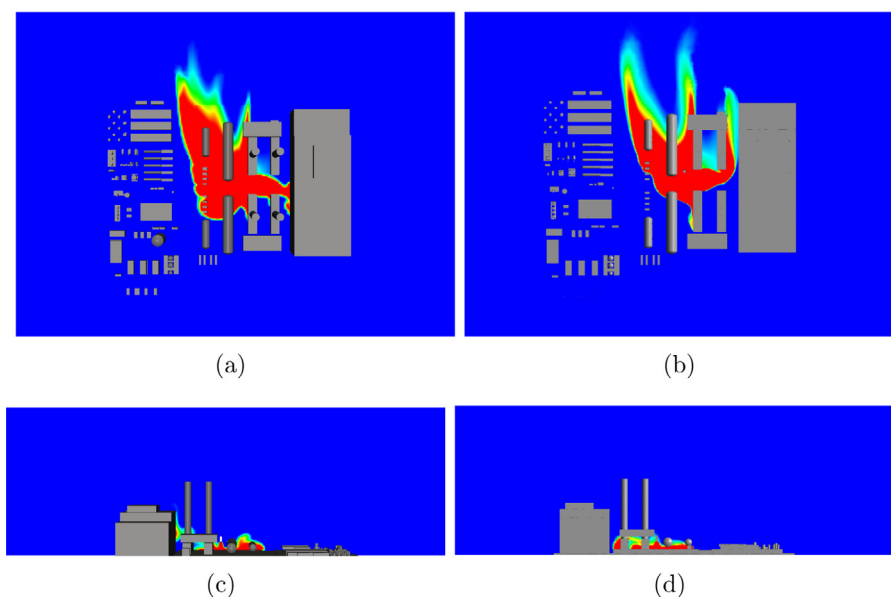


Fig. 17 – Flammable gas cloud contours. Wind is blowing from South and leak points downwards. (a) Top sectional view – OpenFOAM, (b) Top sectional view – ANSYS-CFX, (c) Side sectional view – OpenFOAM, (d) Side sectional view – ANSYS-CFX. All plots at 300 s.

highlight that different meshes were used for each code. The same trend is noticed from the side view analysis (Fig. 14(c) and (d)).

The iso-contour ranges from 0.05 to 0.15 molar fraction of methane. Top view refers to a plane cut at 2.5 m from the deck level. The side view cut is place at $y = 52.0$ m.

Figs. 15–17 address the gas dispersion simulations with leak orientation Down ($-z$) and wind blowing from West, North and South, respectively.

As previously discussed very similar flow patterns are also observed for the cases which the wind is blowing from West, North and South. Qualitative analysis of the plots for x - y view as well as for the plots for z - x view of the cases presented in Figs. 15–17 shows that the gas cloud is spread over the process deck.

4. Conclusions

In this paper we customised the rhoReactingBuoyantFoam solver for gas jet and gas dispersion modelling. The turbulence closure problem was tackled with two different standard models, namely $k-\epsilon$ and SST $k-\omega$. Comparison with experimental data for the gas jet release shown no significant difference between the turbulence models. Excellent agreement with experimental data is observed in the near field. Subsonic and sonic releases were considered, and in both cases, the modelling proposed in this work agreed well with experimental data. As far as the sonic release is concerned, the effect of the expansion zone is well captured at the near field zone of the release and the numerical findings are also in good agreement with the experimental data. The Courant

number influence was also investigated and solver presented good stability when high Courant numbers were considered.

A jet case conducted by Gant and Ivings (2005) was reproduced with success and the plume generated presented very little difference, and it was within 10% in volume.

An engineering case study was conducted considering standard ANSYS-CFX setup and the modelling proposed in the current work. Good agreement was observed between the open source and the commercial code. The greatest difference between the two codes is within 7% in volume for the case that presented the largest cloud. Overall the difference among the cloud sizes are calculated are within 20%.

Both codes are robust. The OpenFOAM has greater versatility as it is an open source and free tool. On the other hand, there is still no solver designed to run gas dispersion in steady state mode. The ANSYS CFX, however, can deal with steady state simulations for gas dispersion. On the other hand, the source code is not available and it requires the license purchase.

The utilisation of rhoReactingBuoyantFoam solver from OpenFOAM package appears as a promising free alternative for gas dispersion simulation. However, the influence of the various numerical schemes available in the code is worth investigating as future work.

The detailed setup of the gas dispersion modelling is presented in appendix A. It enables the modelling of accidental gas releases. The customised version of the code can be applied for optimisation of gas detectors, flammable cloud volume calculation for explosion modelling purposes, area classification, toxic gas releases and vulnerability analysis within the same level of confidence of ANSYS-CFX.

Acknowledgements

Thanks are due to CENAPAD-SP (National Centre for High Performance Computing in Sao Paulo, Brazil), Campinas/FINEP – MCT project.

Appendix A. Engineering procedure for gas dispersion simulation in OpenFOAM

This appendix section presents in detail the set up and the necessary modification in order to run gas dispersion scenarios using the rhoReactionBuoyantFoam solver.

The main aspect of the customisation process lies on the modification of the various properties and the correct switch off of the combustion model.

The modifications are explained below.

Be aware that the set up presented in this section does not necessary reflect what has been used in each individual simulation. It serves as a guideline for the reader.

A.1. Basic properties setup

The user can use the files located in the combustion>reactingFoam folder as the starting point for the gas dispersion modelling. All files in the constant folder must be edited by following the sequence of Fig. A18.

Table A6 – chemistryProperties.

Variable	reactingFoam	rhoReactingBuoyantFoam
chemistryThermo	psi	rho
chemistry	on	off

Table A7 – combustionProperties.

Variable	reactingFoam	rhoReactingBuoyantFoam
combustionModel	laminar <psi-ChemistryCombustion>	noCombustion <rhoThermoCombustion>
active	true	
	laminarCoeffs	noCombustionCoeffs

Table A8 – Acceleration of gravity (g).

Variable	reactingFoam	rhoReactingBuoyantFoam
value	(0 0 0)	(0 0 –9.8)

Table A9 – Reactions.

Variable	reactingFoam	rhoReactingBuoyantFoam
Species	O ₂	O ₂
	methaneReaction { type irreversibleArrheniusReaction; reactions reaction "CH ₄ + 2O ₂ = CO ₂ { } + 2H ₂ O"; A 5.2e16; beta 0; Ta 14906; }	

Table A10 – thermophysicalProperties.

Variable	reactingFoam	rhoReactingBuoyantFoam
Type	hePsiThermo	heRhoThermo
inertSpecie	N ₂	N ₂ O ₂ CH ₄

The first step is to edit the *ChemistryProperties* file. The calculation method used by the thermodynamic package should be modified as shown in Table A6.

Table A7 addresses the modifications in the *combustionProperties* file.

The force of gravity that is not accounted for in the reactingFoam solver must be considered when modelling gas dispersion scenarios. In order to account for the buoyancy effects it is important to modify the g file as shown in Table A8.

Besides deactivation of the combustion model it is also required that the chemical species not involved in the chemical reaction are removed. The removal of the chemical species can be done as described in Table A9. This action is properly done in the file *reactions*.

It is also important to bear in mind that the edition of the *thermo.compressibleGas* file is also required. Data from CO₂ e H₂O of janaf and sutherland models must be removed.

In the *thermophysicalProperties* file modify the type variable and the inertSpecie as shown in Table A10.

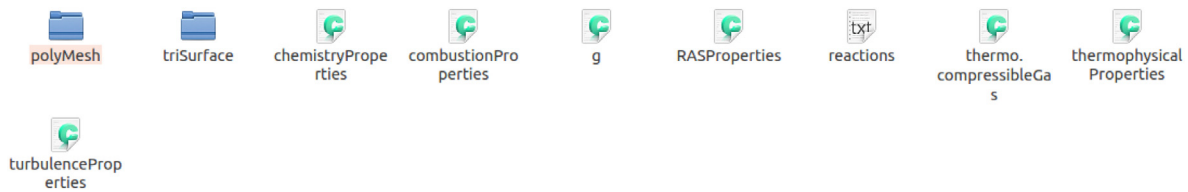


Fig. A18 – Files containing the properties and that should be modified.

Table A11 – turbulenceProperties.

Variable	reactingFoam	rhoReactingBuoyantFoam
simulationType	laminar	RASModel

Table A12 – RASProperties.

Variable	rhoReactingBuoyantFoam	
RASModel	kEpsilon	kOmegaSST
turbulence	on	on
printCoeffs	on	on

```
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    location     "system";
    object       fvSchemes;
}
// *****

ddtSchemes
{
    default      Euler;
}

gradSchemes
{
    default      Gauss linear;
}

divSchemes
{
    default      none;

    div(phi,U)      Gauss upwind;
    div(phi,Yi_h)   Gauss upwind;
    div(phi,K)       Gauss upwind;
    div(phiid,p)     Gauss upwind;
    div(phiid,p_rgh) Gauss upwind;
    div(phi,epsilon) Gauss upwind;
    div(phi,omega)   Gauss upwind;
    div(phi,k)       Gauss upwind;
    div((muEff*dev2(T(grad(U)))) Gauss linear;
}

laplacianSchemes
{
    default      Gauss linear corrected;
}

interpolationSchemes
{
    default      linear;
}

snGradSchemes
{
    default      corrected;
}

fluxRequired
{
    default      no;
    p_rgh;
}
// *****
```

Fig. A19 – fvSchemes file.

The *turbulenceProperties* file must be modified in order to take turbulence modelling into account. The user should follow the setup presented in Table A11.

Thus, at the case should be added turbulence model type for RANS approach as shown in Table A12.

A.2. Numerical setup and boundary conditions

The *system* folder contains the files that control the simulation. In this directory the files *fvSchemes* and *fvSolution* should be customised.

```
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    location     "system";
    object       fvSolution;
}
// *****

solvers
{
    "(rho|rhoFinal)"
    {
        solver          PCG;
        preconditioner   DIC;
        tolerance        1e-6;
        relTol           0;
    };

    p_rgh
    {
        solver          GAMG;
        tolerance        1e-5;
        relTol           0.01;
        smoother         GaussSeidel;
        cacheAgglomeration true;
        nCellsInCoarsestLevel 10;
        agglomerator      faceAreaPair;
        mergeLevels       1;
    };

    p_rghFinal
    {
        Sp_rgh;
        tolerance        1e-6;
        relTol           0;
    };

    "(U|Yi|k|h|epsilon|omega)"
    {
        solver          PBICG;
        preconditioner   DILU;
        tolerance        1e-6;
        relTol           0.1;
        nSweeps          1;
    };

    "(U|Yi|k|h|epsilon|omega)Final"
    {
        $U;
        tolerance        1e-6;
        relTol           0;
    };
}

PIMPLE
{
    momentumPredictor yes;
    nOuterCorrectors 1;
    nCorrectors 2;
    nNonOrthogonalCorrectors 0;
}
// *****
```

Fig. A20 – fvSolution file.

In first file the user must be add the turbulence and *p_rgh* resolution criterion. In the *fvSolution* file the matrix method for resolution of the system of equations is selected. It is important to have in mind that the same information added must be consistent with the information inserted in the *fvSchemes* file. As shown in Figs. A19 and A20.

Regarding the boundary condition, all files located in the 0 (zero) folder must be in accordance with Figs. A21–A24.

Appendix B. OpenFOAM V3.0

The OpenFOAM version 3.0.0 requires the following adjustments:


```

// *****
//mut
dimensions      [1 -1 -1 0 0 0];
internalField    uniform 0.0;
boundaryField
{
    wall
    {
        type            mutkWallFunction;
        Cmu              0.09;
        kappa            0.41;
        E                9.8;
        value            uniform 0;
    }
    inlet
    {
        type            calculated;
        value            uniform 0;
    }
    east
    {
        type            calculated;
        value            uniform 0;
    }
    open
    {
        type            calculated;
        value            uniform 0;
    }
}

// *****
// *****
//alphanat
dimensions      [1 -1 -1 0 0 0];
internalField    uniform 0;
boundaryField
{
    wall
    {
        type            compressible::alphanatWallFunction;
        Prt              0.85;
        value            uniform 0;
    }
    inlet
    {
        type            calculated;
        value            uniform 0;
    }
    east
    {
        type            calculated;
        value            uniform 0;
    }
    open
    {
        type            calculated;
        value            uniform 0;
    }
}

// *****
// *****
//Ydefault
dimensions      [0 0 0 0 0 0];
internalField    uniform 0.0;
boundaryField
{
    wall
    {
        type            zeroGradient;
    }
    inlet
    {
        type            fixedValue;
        value            uniform 0.0;
    }
    open
    {
        type            inletOutlet;
        inletValue        uniform 0.0;
        value              uniform 0.0;
    }
    east
    {
        type            fixedValue;
        value            uniform 0.0;
    }
}

// *****
// *****
//CH4
dimensions      [0 0 0 0 0 0];
internalField    uniform 0.0;
boundaryField
{
    wall
    {
        type            zeroGradient;
    }
    inlet
    {
        type            fixedValue;
        value            uniform 1.0;
    }
    east
    {
        type            fixedValue;
        value            uniform 0.0;
    }
    open
    {
        type            inletOutlet;
        inletValue        uniform 0.0;
        value              uniform 0.0;
    }
}

// *****
// *****

```

Fig. A21 – Boundary condition files – mut, Ydefault, alphanat, CH4.

Folder 0

The viscosity file **mut** is no longer used. It may be replaced by the **nut** file (refer to Fig. B25):

Folder constant

The **RASProperties** file is no longer used. The information from the **RASProperties** file is now merged into a single file, namely **turbulenceProperties** as presented in Fig. B26.

There is also an additional file (**transportProperties**) that must be added to the constant folder (refer to Fig. B27).

Folder system

A set of wall distance algorithms is used in the turbulence modelling. If the user is using the k-omegaSST turbulence model, for instance, the following line must be added at the bottom of the **fvSchemes** file

```

wallDist
{
    method meshWave;
}

```

The file we have tested using the k-omegaSST turbulence model looks like Fig. B28.

```

// *****
//N2
dimensions      [0 0 0 0 0 0];
internalField    uniform 0.79;
boundaryField
{
    wall
    {
        type      zeroGradient;
    }
    inlet
    {
        type      fixedValue;
        value      uniform 0.0;
    }
    east
    {
        type      fixedValue;
        value      uniform 0.79;
    }
    open
    {
        type      inletOutlet;
        inletValue uniform 0.79;
        value      uniform 0.79;
    }
}
|
// *****
// *****
//T
dimensions      [0 0 0 1 0 0];
internalField    uniform 297;
boundaryField
{
    wall
    {
        type      zeroGradient;
    }
    inlet
    {
        type      fixedValue;
        value      uniform 297;
    }
    east
    {
        type      fixedValue;
        value      uniform 297;
    }
    open
    {
        type      inletOutlet;
        inletValue uniform 297;
        value      uniform 297;
    }
}
// *****
// *****
//O2
dimensions      [0 0 0 0 0 0];
internalField    uniform 0.21;
boundaryField
{
    wall
    {
        type      zeroGradient;
    }
    inlet
    {
        type      fixedValue;
        value      uniform 0.0;
    }
    east
    {
        type      fixedValue;
        value      uniform 0.21;
    }
    open
    {
        type      inletOutlet;
        inletValue uniform 0.21;
        value      uniform 0.21;
    }
}
// *****
//U
dimensions      [0 1 -1 0 0 0];
internalField    uniform (-6 0 0);
boundaryField
{
    wall
    {
        type      fixedValue;
        value      uniform (0 0 0);
    }
    inlet
    {
        type      fixedValue;
        value      uniform (0 0 -100);
    }
    east
    {
        type      fixedValue;
        value      uniform (-6 0 0);
    }
    open
    {
        type      pressureInletOutletVelocity;
        value      phi;
        phi        $internalField;
    }
}
// *****

```

Fig. A22 – Boundary condition files – N₂, O₂, T and U.

```

// ***** // *****
//p                                     //p_rgh
dimensions      [1 -1 -2 0 0 0];      dimensions      [1 -1 -2 0 0 0];
internalField    uniform 101325;         internalField    uniform 101325;
boundaryField
{
    wall
    {
        type          calculated;
        value          $internalField;
    }
    inlet
    {
        type          calculated;
        value          $internalField;
    }
    east
    {
        type          calculated;
        value          $internalField;
    }
    open
    {
        type          calculated;
        value          $internalField;
    }
}

// ***** // *****
//epsilon                               //k
dimensions      [0 2 -3 0 0 0];         dimensions      [0 2 -2 0 0 0];
internalField    uniform 0.1;             internalField    uniform 0.01;
boundaryField
{
    inlet
    {
        type          zeroGradient;
    }
    wall
    {
        type          compressible::epsilonWallFunction;
        value          $internalField;
    }
    open
    {
        type          zeroGradient;
    }
    east
    {
        type          zeroGradient;
    }
}

// ***** // *****

```

Fig. A23 – Boundary condition files – p, p_rgh, epsilon and k.

```

// *****
//omega
dimensions      [0 0 -1 0 0 0];
internalField    uniform 0.1;
boundaryField
{
    wall
    {
        type          compressible::omegaWallFunction;
        value          uniform 0.1;
        Cmu            0.09;
        kappa           0.41;
        E              9.8;
    }
    inlet
    {
        type zeroGradient;
    }
    east
    {
        type zeroGradient;
    }
    open
    {
        type zeroGradient;
    }
}

// *****

```

Fig. A24 – Boundary condition file – omega.

```

/*-----C++-----*\
|=====|
| \ / F i e l d | OpenFOAM: The Open Source CFD Toolbox |
| \ / O p e r a t i o n | Version: v3.0+ |
| \ / A n d | Web: www.OpenFOAM.com |
| \ / M a n i p u l a t i o n |
|-----*/
FoamFile
{
    version 2.0;
    format ascii;
    class volScalarField;
    location "0";
    object nut;
}
// *****

dimensions [0 2 -1 0 0 0];

internalField uniform 0;

boundaryField
{
    inlet
    {
        type calculated;
        value uniform 0;
    }
    open
    {
        type calculated;
        value uniform 0;
    }
    wall
    {
        type nutkWallFunction;
        value uniform 0;
    }

    north
    {
        type calculated;
        value uniform 0;
    }
}

```

Fig. B25 – Nut file contents.

```

/*-----C++-----*\
|=====|
| \ / F i e l d | OpenFOAM: The Open Source CFD Toolbox |
| \ / O p e r a t i o n | Version: v3.0+ |
| \ / A n d | Web: www.OpenFOAM.com |
| \ / M a n i p u l a t i o n |
|-----*/
FoamFile
{
    version 2.0;
    format ascii;
    class dictionary;
    location "constant";
    object turbulenceProperties;
}
// *****

simulationType RAS;

RAS
{
    RASModel kOmegaSST;

    turbulence on;

    printCoeffs on;
}
// *****

```

Fig. B26 – turbulenceProperties file contents.

```

~ /*-----C++-----*\
|=====|
| \ / F i e l d | OpenFOAM: The Open Source CFD Toolbox |
| \ / O p e r a t i o n | Version: v3.0+ |
| \ / A n d | Web: www.OpenFOAM.com |
| \ / M a n i p u l a t i o n |
|-----*/
FoamFile
{
    version 2.0;
    format ascii;
    class dictionary;
    location "constant";
    object transportProperties;
}
// *****

transportModel Newtonian;

nu [0 2 -1 0 0 0] 1e-05;

// *****
~

```

Fig. B27 – transportProperties file contents.

```

/*-----C++-----*\
|=====|
| \ / F i e l d | OpenFOAM: The Open Source CFD Toolbox |
| \ / O p e r a t i o n | Version: v3.0+ |
| \ / A n d | Web: www.OpenFOAM.com |
| \ / M a n i p u l a t i o n |
|-----*/
FoamFile
{
    version 2.0;
    format ascii;
    class dictionary;
    location "system";
    object fvSchemes;
}
// *****

ddtSchemes
{
    default Euler;
}

gradSchemes
{
    default Gauss linear;
}

divSchemes
{
    default none;

    div(phi,U) Gauss upwind;
    div(phi,Yi_h) Gauss upwind;
    div(phi,K) Gauss upwind;
    div(phi,p) Gauss upwind;
    div(phi,p_ggh) Gauss upwind;
    div(phi,epsilon) Gauss upwind;
    div(phi,omega) Gauss upwind;
    div(phi,k) Gauss upwind;
    div(((rho*muEff)*dev2(T(grad(U)))) Gauss linear;
}

laplacianSchemes
{
    default Gauss linear corrected;
}

interpolationSchemes
{
    default linear;
}

snGradSchemes
{
    default corrected;
}

wallDist
{
    method meshWave;
}

fluxRequired
{
    default no;
    p_rgh;
}

```

Fig. B28 – fvSchemes file contents.

References

- Benintendi, R., 2010. Turbulent jet modelling for hazardous area classification. *J. Loss Prevent. Proc.*, 373–378.
- Birch, A.D., Brown, D.R., Dodson, M.G., Swaffield, F., 1984. The structure and concentration decay of high pressure jets of natural gas. *Comb. Sci. Technol.* 36, 249–261.
- Birch, A.D., Huges, D.J., Swaffield, F., 1987. Velocity decay of high pressure jets. *Comb. Sci. Technol.* 52, 161–171.
- Chuech, S.G., Lai, M.C., Faeth, G.M., 1989. Structure of turbulent Sonic underexpanded free jets. *Am. Inst. Aeronaut. Astronaut. J.* 27 (5), 549–559.
- Cormier, B.R., Qi, R., Yun, G., Zhang, Y., Mannan, M.S., 2009. Application of computational fluid dynamics for LNG vapor dispersion modeling: a study of key parameters. *J. Loss Prevent. Proc.*, 316–323.
- Dixon, C., 2012. Gas dispersion with OpenFOAM. Shell Research Ltd <http://ukelg.ps.ic.ac.uk/49CD.pdf>.
- Ferreira Jr., E.S., Vianna, S.S.V., 2014. A novel free and advanced large eddy simulation computational fluid dynamics tool for gas dispersion. *Int. J. Model. Simul. Petr. Ind.* 8, 1–6.

- Ferreira, T.D., Vianna, S.S.V., 2014. A Novel Coupled Response Surface for Flammable Gas Cloud Volume Prediction. *Int. J. Model. Simul. Petr. Ind.* 8 (1).
- Ferreira Jr., E.S., Vianna, S.S.V., 2016. Large eddy simulation combined to equivalent diameter for turbulent jet modelling and gas dispersion. *Braz. J. Chem. Eng. Print.*
- Gant, S.E., Ivings, M.J., 2005. CFD Modelling of Low Pressure Jets for Area Classification. Health and Safety Laboratory.
- Hanna, S.R., Chang, J.C., 2001. Use of the Kit Fox field data to analyze dense gas dispersion modeling issues. *Atmos. Environ.*, 2231–2242.
- Hanna, S.R., Hansen, O.R., Dharmavaram, S., 2004. FLACS CFD air quality model performance evaluation with Kit Fox, MUST, Prairie Grass, and EMU observations. *Atmos. Environ.*, 4675–4687.
- Hansen, O.R., Gavelli, F., Ichard, M., Davis, S.G., 2010. Validation of FLACS against experimental data sets from the model evaluation database for LNG vapor dispersion. *J. Loss Prevent. Proc.*, 857–877.
- Ivings, M.J., Gant, S.E., Saunders, C.J., Pocock, D.J., 2010. Flammable gas cloud build up in a ventilated enclosure. *J. Hazard. Mater.*, 170–176.
- Kumaresh, G.R., Fossan, I., Venkatraman, M.M., Giljarhus, K.E., Spangelo, Ø., Jensen, S., 2016. CFD-based transient ignition probability modeling of gas leaks in enclosures. *Int. J. Comp. Methods* 13 (2).
- Mack, A., Spruijt, M.P.N., 2013. Validation of OpenFoam for heavy gas dispersion applications. *J. Hazard. Mater.* 262, 504–516.
- Mouilleau, Y., Champassith, A., 2009. CFD simulations of atmospheric gas dispersion using the Fire Dynamics Simulator (FDS). *J. Loss Prevent. Proc.*, 316–323.
- Qiao, A., Zhang, S., 2010. Advanced CFD modeling on vapor dispersion and vapor cloud explosion. *J. Loss Prevent. Proc.*, 843–848.
- Rigas, F., Sklavounos, S., 2004. Validation of turbulence models in heavy gas dispersion over obstacles. *J. Hazard. Mater. A* 108, 9–20.
- Rocco, S., Woods, A.W., 2015. Dispersion in two-dimensional turbulent buoyant plumes. *J. Fluid Mech.* 774.
- Tauseef, S.M., Rashtchian, D., Abbasi, T., Abbassi, S.A., 2011. A method for simulation of vapour cloud explosions based on computational fluid dynamics (CFD). *J. Loss Prevent. Proc.*, 638–647.
- Wakes, S.J., Holdø, A.E., Meares, A.J., 2002. Experimental investigation of the effect orifice shape and fluid pressure has on high aspect ratio cross-sectional jet behaviour. *J. Hazard. Mater. A89*, 1–27.
- Wilkening, H., Baraldi, D., Heitsch, M., 2008. CFD simulations of light gas release and mixing in the BMC with CFX. *Nucl. Eng. Des.* 238, 618–626.